

# Intelligent Candidate Discovery & Ranking

Ranking candidates the way a great recruiter would — by understanding the role, not matching keywords.

|                              |                                                                          |
|------------------------------|--------------------------------------------------------------------------|
| <b>Challenge</b>             | Redrob / India Runs — Senior AI Engineer (Founding Team)                 |
| <b>Pool</b>                  | 100,000 candidates → trusted top-100 shortlist                           |
| <b>Constraint</b>            | CPU-only · no network · ≤16 GB · ≤5 min at rank time                     |
| <b>Our rank-time runtime</b> | ~150 s on the full 100K pool · 2.9 GB peak RAM · 0% honeypots in top-100 |

## THE PROBLEM

# Keyword filters miss the right person

The JD says it explicitly: *“The right answer is not finding candidates whose skills section contains the most AI keywords. That’s a trap we’ve built into the dataset.”*

### What the JD SAYS

embeddings, vector DB, RAG, NDCG, Pinecone...

### What the JD MEANS

shipped a ranking / search / recommender system to real users at a **product** company; understood retrieval before it was fashionable; is actually **reachable**.

Our design reads the gap between the two — and the dataset is full of traps that punish anyone who doesn’t:

- **Keyword stuffers** — non-engineering titles (Marketing Manager) with packed AI skill lists.
- **Plain-language Tier-5s** — strong engineers who never use the buzzwords.
- **Behavioral twins** — identical profiles differing only in availability signals.
- **~80 honeypots** — arithmetically impossible profiles, forced to tier 0 (>10% in top-100 = instant DQ).

## A transparent hybrid scorer — every point is explainable

| 1 · SEMANTIC                                                                                                                | 2 · FEATURES                                                                                                                                                               | 3 · TRAPS                                                                                                                | 4 · SCORE                                                                                                                                                                      | 5 · REASONING                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| TF-IDF (1–2 gram) of each career narrative · cosine vs a hand-written <b>JD-intent query</b> . CPU-only, no model download. | Interpretable signals: 4 must-have areas (terms + plain-language evidence), role-fit, applied-ML yrs, experience/location fit, product-vs-services, behavioral multiplier. | Honeypot arithmetic checks · keyword-stuffer detector · research-only / LangChain-only / job-hop / CV-without-NLP flags. | $\text{score} = \text{base\_fit} \times \text{behavioral} \times \text{location} - \text{penalties}$ . Honeypots $\rightarrow 0$ . Bounded $[0,1]$ , non-increasing with rank. | 1–2 sentences, <b>extractive</b> from real fields only (no hallucination). Tone tracks the rank; concerns stated honestly. |

**Why no LLM at rank time?** The spec bans network/hosted-LLM calls during ranking, and the JD itself warns that an LLM-per-candidate system can't scale to a 200K pool. A per-candidate GPT call also can't fit the 5-min CPU budget and would fail Stage 3. An optional *offline open-source embedding* index (`precompute_embeddings.py`) can be dropped in via - - cache with no network at rank time.

## THE KEY IDEA

# Dual detection: terminology OR plain-language evidence

Each of the 4 must-have skill areas is satisfied if EITHER the explicit term appears OR plain-language evidence of the same work appears in career descriptions — which buzzword-stuffers can't fake.

| Must-have area            | Explicit terms (easy to stuff)                         | Plain-language evidence (hard to fake)                                                                 |
|---------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Embeddings / retrieval    | sentence-transformers, BGE, E5, RAG, vector search     | "built a recommendation / relevance / matching system",<br>"connect users to what they're looking for" |
| Vector DB / hybrid search | Pinecone, Weaviate, Qdrant, FAISS, Elasticsearch, HNSW | "search index / infrastructure", "nearest-neighbour over a large dataset"                              |
| Strong Python engineering | python, pytorch, fastapi, pyspark, numpy               | (engineering career titles required)                                                                   |
| Ranking evaluation        | NDCG, MRR, MAP, A/B testing, learning-to-rank          | "compared model versions before rollout", "measured click-through / ranking quality"                   |

**Result: a Tier-5 engineer who built a ranking system at a product company but never wrote "RAG" still reaches the top 10; a "Marketing Manager" with 8 AI skills listed does not.**

## SCORING

### Weighted base-fit, availability multiplier, JD-driven penalties

| Base-fit weight               | %    | Penalty (JD 'do NOT want')           | weight |
|-------------------------------|------|--------------------------------------|--------|
| semantic (TF-IDF)             | 0.22 | keyword stuffer                      | 0.45   |
| role-fit (title)              | 0.18 | no engineering career                | 0.30   |
| must-have coverage            | 0.14 | research-only, no production         | 0.22   |
| ranking/retrieval evidence    | 0.12 | services-only ( $\times 1$ -product) | 0.20   |
| applied-ML years              | 0.09 | job-hopping                          | 0.18   |
| experience fit                | 0.08 | recent LangChain-only                | 0.15   |
| evaluation + NLP/IR + shipped | 0.13 |                                      |        |
| product + assessment + edu    | 0.04 | Honeypot $\rightarrow$ score = 0     | DQ     |

**Behavioral signals multiply, they don't add** — a perfect-on-paper candidate inactive 6 months with a 5% response rate is, for hiring, unavailable. The multiplier (0.45–1.15) folds in recency, recruiter-response, open-to-work and notice period. The services penalty respects the JD carve-out: prior product-company depth shrinks it.

## EVALUATION

# Offline harness + ablation — we don't fly blind

The JD lists 'designing evaluation frameworks for ranking systems' as a must-have, and the leaderboard is hidden. So we built a transparent JD-derived proxy oracle (tiers 0–5) and run the exact competition composite + a feature-ablation study. **The proxy shares feature code with the ranker — it is a self-consistency check, not ground truth**; the ablation deltas are the real signal.

| Remove this component → | proxy-composite | $\Delta$ vs baseline | overlap@10 | reads as                               |
|-------------------------|-----------------|----------------------|------------|----------------------------------------|
| (baseline, all on)      | 0.981           | —                    | 1.00       | —                                      |
| behavioral multiplier   | 0.768           | −0.213               | 0.40       | critical — availability drives quality |
| penalties               | 0.922           | −0.060               | 0.70       | important — filters 'do-not-want'      |
| location fit            | 0.979           | −0.003               | 0.40       | reshuffles top-10 toward India tier-1  |
| semantic layer          | 0.989           | +0.008               | 0.80       | features already capture most signal   |

| Trap robustness (top-100) | honeypots | keyword-stuffers | consulting-heavy |
|---------------------------|-----------|------------------|------------------|
| rate                      | 0.0%      | 0.0%             | 1.0%             |

## RESULT

### Top-10 — exactly the profiles the JD describes

| #  | Title                           | Company     | YoE | Location  |
|----|---------------------------------|-------------|-----|-----------|
| 1  | Senior AI Engineer              | Meta        | 7.9 | Noida     |
| 2  | Senior ML Engineer              | Zomato      | 7.2 | Noida     |
| 3  | Lead AI Engineer                | Razorpay    | 6.7 | Jaipur    |
| 4  | Machine Learning Engineer       | Verloop.io  | 5.7 | Gurgaon   |
| 5  | Recommendation Systems Engineer | CRED        | 8.0 | Noida     |
| 6  | Search Engineer                 | Sarvam AI   | 7.6 | Gurgaon   |
| 7  | Recommendation Systems Engineer | Amazon      | 6.5 | Pune      |
| 8  | Senior ML Engineer              | Niramai     | 7.9 | Bangalore |
| 9  | AI Engineer                     | Rephrase.ai | 6.8 | Delhi     |
| 10 | Recommendation Systems Engineer | Microsoft   | 5.9 | Delhi     |

Product companies · ranking/search/recsys titles · 5–8 yrs · India tier-1 cities — and 0 honeypots.

Reproduce (single command — Stage 3):

```
pip install -r requirements.txt
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
python validate_submission.py --submission ./submission.csv --candidates ./candidates.jsonl
```